ASSESSMENT TOOL 2 – Design and Development Model

Course Code: MLA03

Course Name: Reinforcement Learning for Environment and Agent

Name: K. L. Khaviya varshini Reg. no.: 192472381

Model-Based Data Generation

QUESTION 1:

The actual process is:

Real Environment → Real Experiences → Learn Environment Model → Generate Synthetic Experiences → Use Synthetic Data for Training → Improved Policy

It also shows the important feedback loop: the improved policy interacts with the environment again.

CODE:

```
state = 2

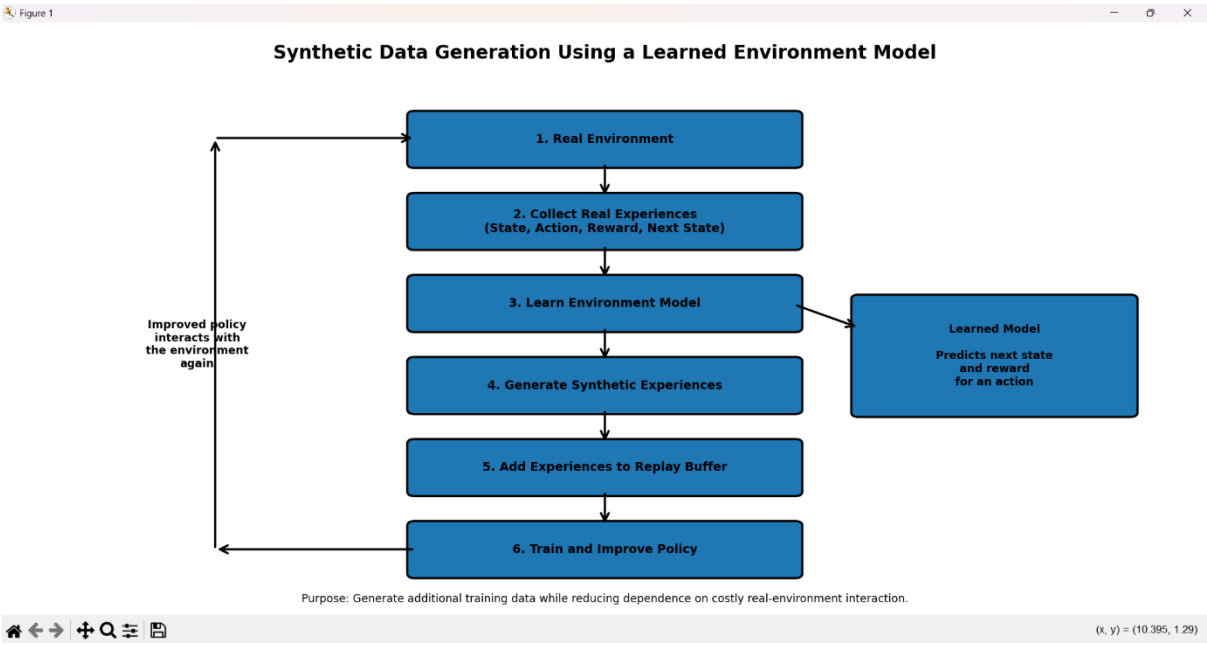
action = 1

next_state = state + action

reward = 1 if next_state >= 5 else 0

synthetic_experience = (state, action, reward, next_state)

print("Synthetic Experience:", synthetic_experience)
```



QUESTION 2:

Relationship

Environment Model
→ generates Experiences

Generated Experiences
→ stored in Replay Buffer

Replay Buffer
→ provides data for Policy Learning

Policy Learning
→ produces an Improved Policy

CODE:

```
replay_buffer = []

state = 1

action = 2

next_state = state + action

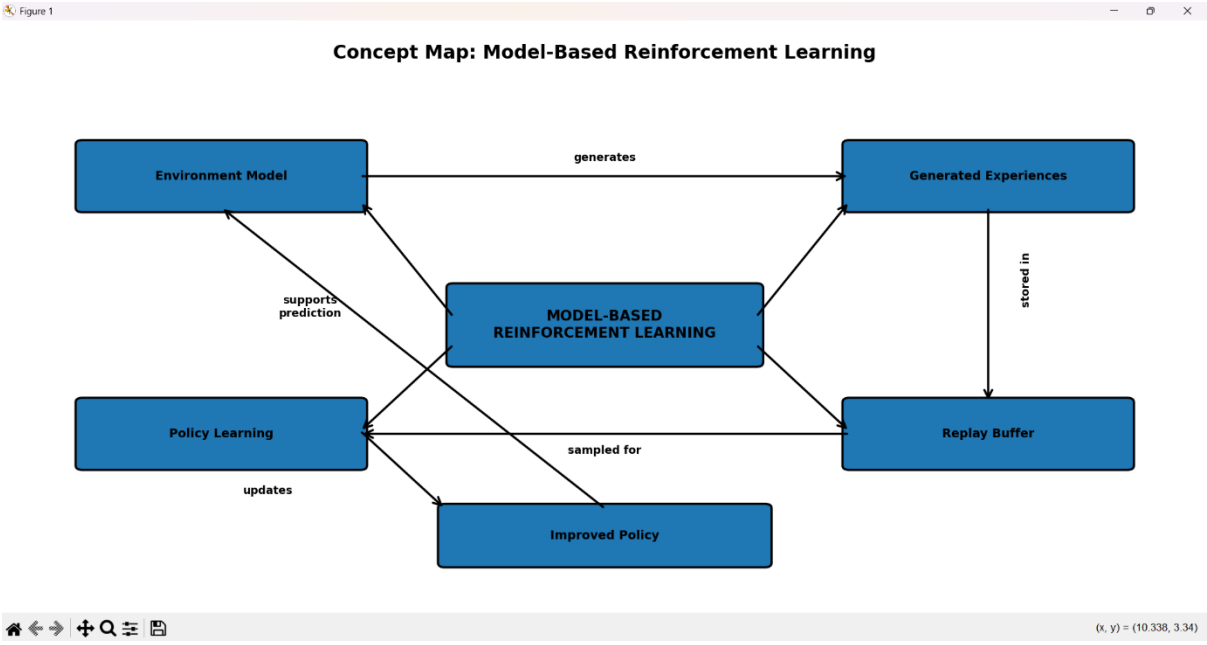
reward = 1

experience = (state, action, reward, next_state)

replay_buffer.append(experience)

print("Replay Buffer:", replay_buffer)

print("Policy learns from stored experiences.")
```

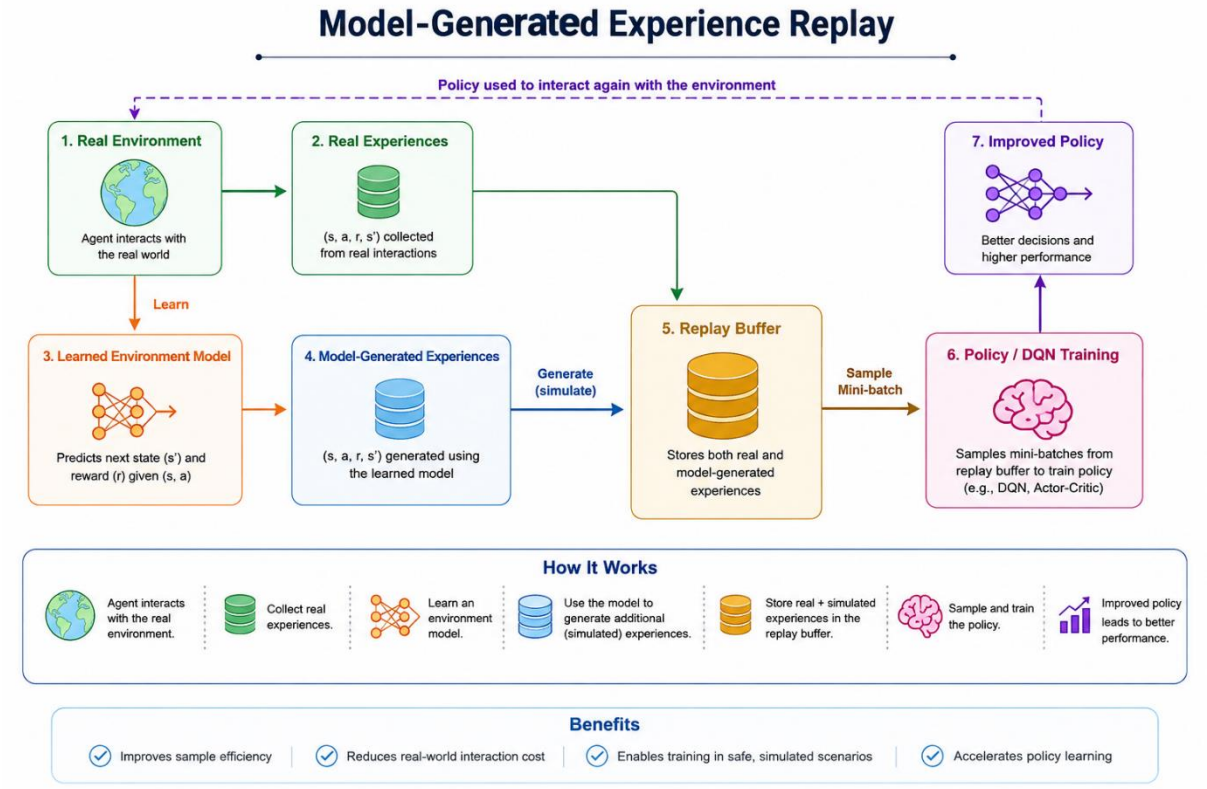


QUESTION 3:

Model-generated experience replay uses a learned environment model to create additional simulated experiences. These experiences are stored in the replay buffer along with real experiences and are used for policy training, improving learning efficiency.

CODE:

```
replay_buffer = []
real_experience = (1, 2, 1, 3)
replay_buffer.append(real_experience)
for i in range(3):
    state = i
    action = 1
    next_state = state + action
    reward = 1 if next_state >= 3 else 0
    replay_buffer.append(
        (state, action, reward, next_state))
print("Replay Buffer:")
for experience in replay_buffer:
    print(experience)
```

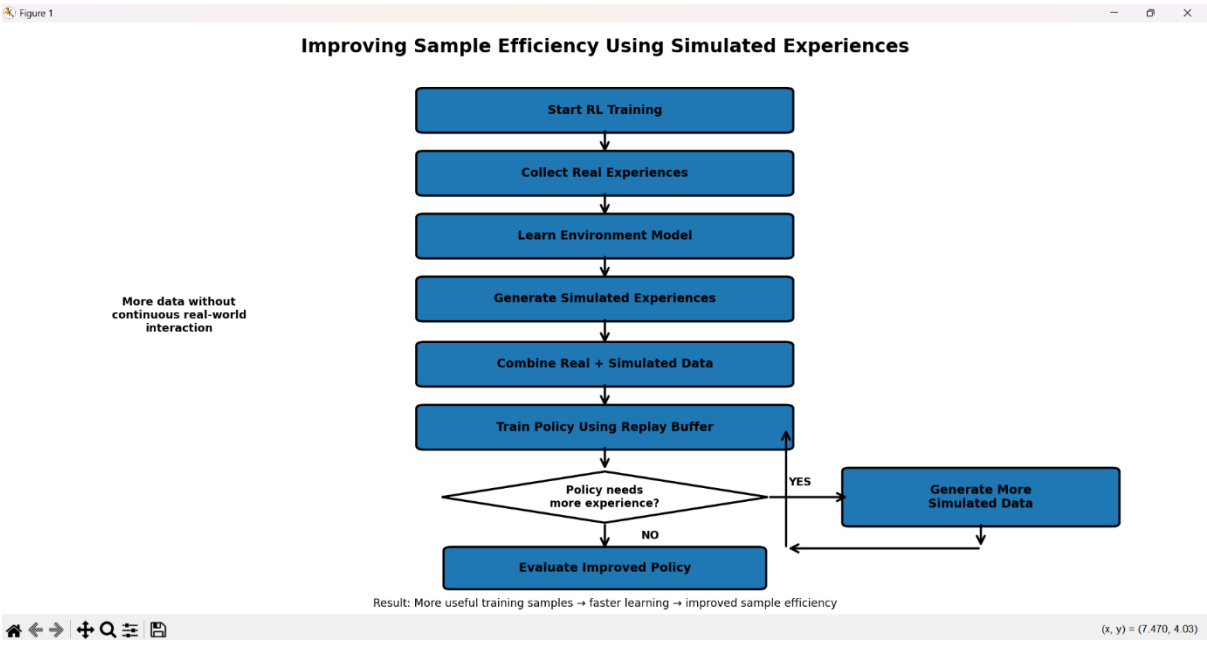


QUESTION 4:

Simulated experiences provide additional training data without requiring continuous interaction with the real environment. This increases the number of experiences available for learning and improves the sample efficiency of the reinforcement learning agent.

CODE:

```
real_data = [
    (0, 1, 0, 1),
    (1, 1, 0, 2)
]
simulated_data = [
    (2, 1, 1, 3),
    (3, 1, 1, 4),
    (1, -1, 0, 0)
]
training_data = real_data + simulated_data
print("Real Experiences:", len(real_data))
print("Simulated Experiences:", len(simulated_data))
print("Total Training Experiences:", len(training_data))
```



QUESTION 5:

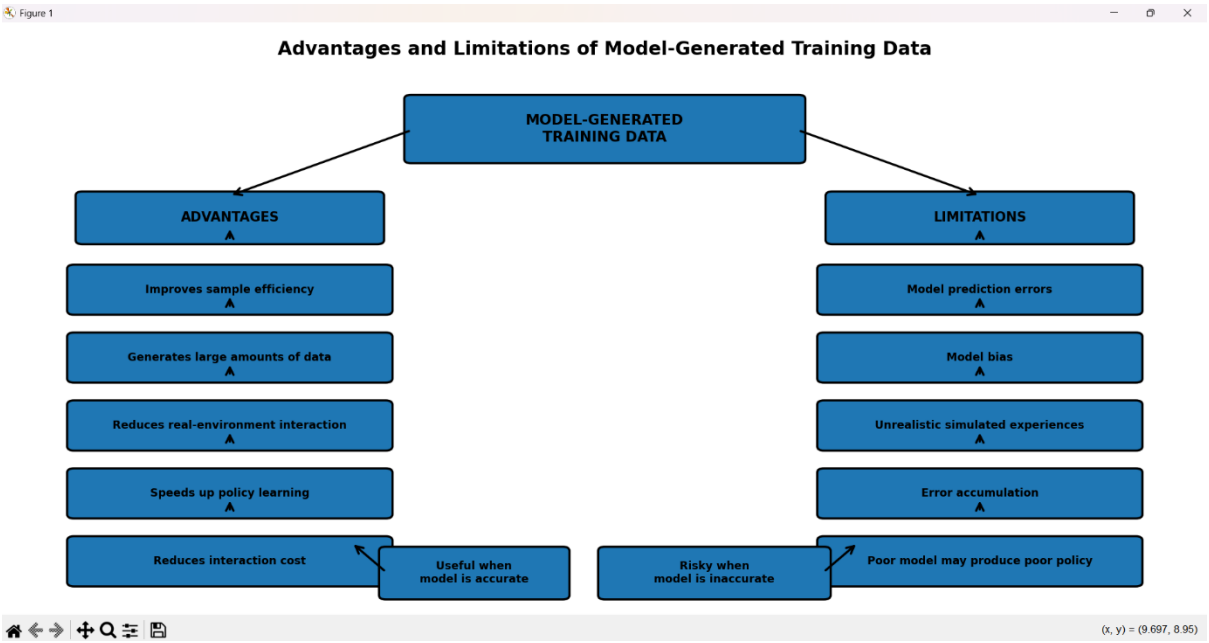
Model-generated training data can improve sample efficiency, reduce real-world interaction costs, and speed up policy learning. However, inaccurate models may introduce bias, unrealistic experiences, and accumulated errors, which can negatively affect the learned policy.

CODE:

```
advantages = ["Improves sample efficiency","Generates additional experiences","Reduces real interactions","Faster policy learning"]

limitations = [
    "Model prediction errors",
    "Model bias",
    "Unrealistic experiences",
    "Error accumulation"
]

print("Advantages:")
for item in advantages:
    print("-", item)
print("\nLimitations:")
for item in limitations:
    print("-", item)
```



QUESTION 6:

A Model-Based Reinforcement Learning agent interacts with the environment and collects state, action, reward, and next-state information. This experience is used to learn an environment model. The learned model supports planning by predicting future outcomes, and the policy is optimized based on these predictions.

CODE:

```
state = 0

for step in range(3):

    action = 1

    next_state = state + action

    reward = 1 if next_state >= 2 else 0

    predicted_next_state = state + action

    if reward > 0:

        action = 1

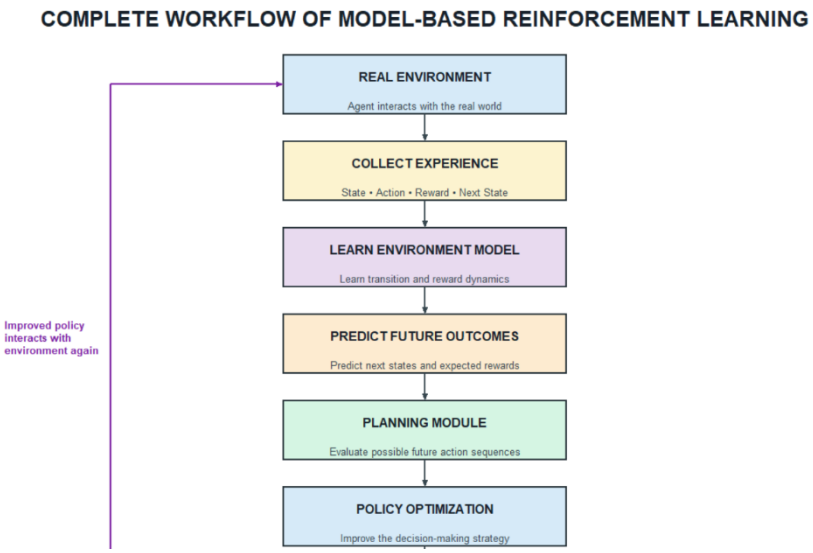
    print("State:", state,

          "Action:", action,

          "Reward:", reward,

          "Next State:", next_state)

    state = next_state
```



QUESTION 7:

In Model-Based Reinforcement Learning, the environment provides states and rewards after the agent performs an action. The environment model learns the transition behavior and predicts possible future states and rewards.

CODE:

```
state = 2

def model(state, action):
    next_state = state + action
    reward = 10 if next_state >= 4 else 0
    return next_state, reward

actions = [-1, 1]

best_action = None

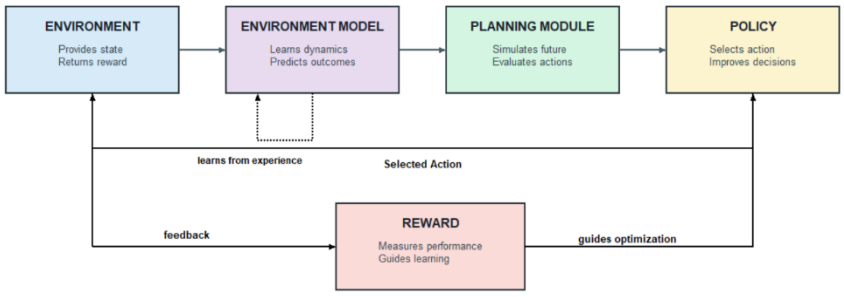
best_reward = -1

for action in actions:
    next_state, reward = model(state, action)
    if reward > best_reward:
        best_reward = reward
        best_action = action

print("Selected Action:", best_action)

print("Expected Reward:", best_reward)
```

CONCEPT MAP: MODEL-BASED REINFORCEMENT LEARNING



QUESTION 8:

The RL agent first gathers experience from the environment and uses this data to learn an environment model. The learned model predicts the next state and reward for different possible actions.

CODE:

```
state = 1

possible_actions = [-1, 1, 2]

def learned_model(state, action):

    next_state = state + action

    reward = next_state * 2

    return next_state, reward

best_action = None

best_reward = float("-inf")

for action in possible_actions:

    next_state, reward = learned_model(state, action)

    print("Action:", action,

          "Predicted State:", next_state,

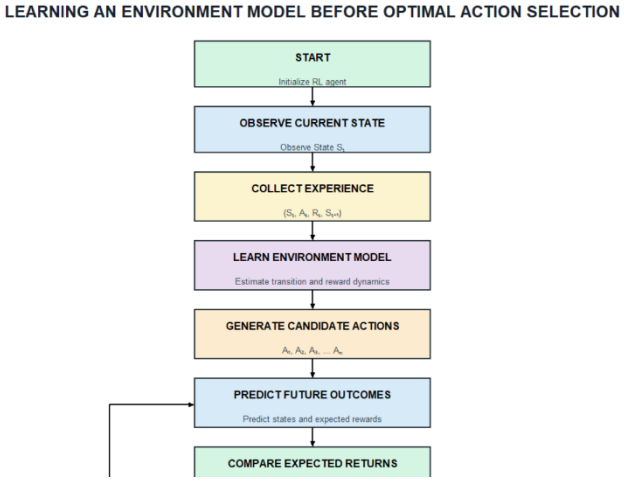
          "Reward:", reward)

    if reward > best_reward:

        best_reward = reward

        best_action = action

print("Optimal Action:", best_action)
```



QUESTION 9:

A Model-Based RL architecture consists of three major components: model learning, planning, and execution. The model-learning component learns the dynamics of the environment from collected experiences.

CODE:

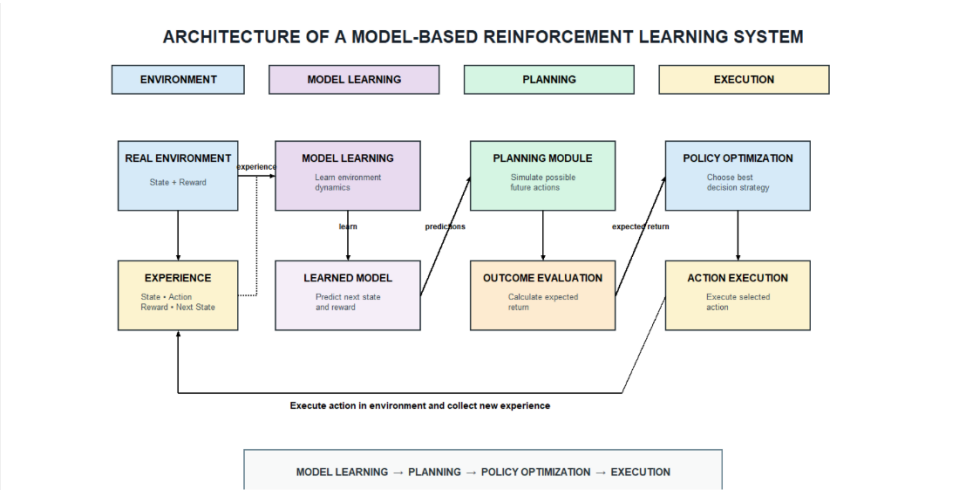
```
state = 0

def model_learning(state, action):
    next_state = state + action
    reward = 1 if next_state >= 3 else 0
    return next_state, reward

def planning(state):
    actions = [-1, 1]
    best_action = None
    best_reward = -1
    for action in actions:
        next_state, reward = model_learning(state, action)
        if reward > best_reward:
            best_reward = reward
            best_action = action
    return best_action

action = planning(state)

next_state, reward = model_learning(state, action), print("Current State:", state), print("Planned Action:", action), print("Next State:", next_state), print("Reward:", reward)
```



QUESTION 10:

Model-Based Reinforcement Learning learns or uses a model of the environment to predict future states and rewards before selecting actions. It supports planning and is generally more sample-efficient, but model learning can increase computational complexity and prediction errors.

